Тема 3: Извлечение данных

Извлечение информации из одной таблицы

Традиционно, весь язык SQL разделяется на три части: язык определения данных DDL (data definition language), язык манипулирования данными DML (data manipulation language) и язык управления данными DCL (data control language). К DDL относятся операторы работы со схемой базы данных, к DML — операторы выборки и изменения данных таблиц, не изменяющие схему, язык DCL состоит, в основном, из операторов управления транзакциями и назначения прав доступа. Основным оператором языка DML является оператор select — извлечение данных из таблиц.

Запись выбранных полей в файл

Оператор SELECT может быть представлен в форме

SELECT <список полей> INTO OUTFILE "имя_файла" FROM <имя таблицы>.

Для использования этой формы команды SELECT необходимы привилегии FILE. Пример записи двух полей таблицы student во внешний файл:

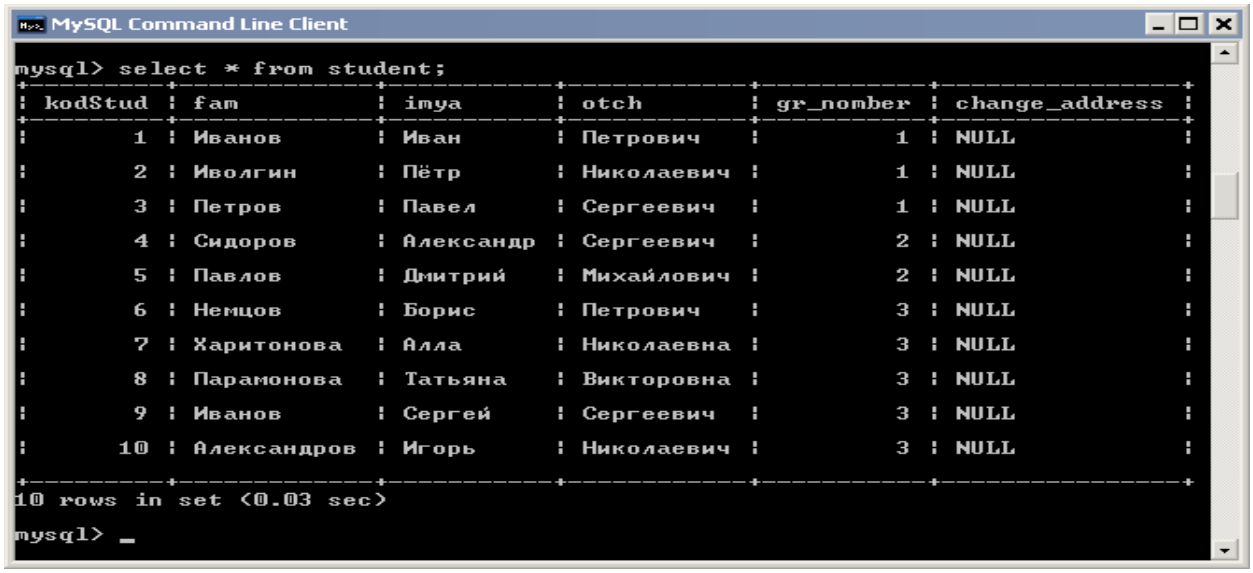
SELECT kodStud, Fam INTO OUTFILE "C:/result.txt" FROM student;

Извлечение всех полей и записей из таблицы

Формат оператора select:

select * from <имя таблицы>

На рис. 3.1 представлен пример извлечения информации из всех полей таблицы student. Как видно из рисунка, столбцы следуют в том же порядке, что и в исходной таблице.



```
MySQL Command Line Client
mysql> select * from student;
+---------+-------------+-----------+------------+-----------+-----------------+
| kodStud | fam         | imya      | otch       | gr_nomber | change_address  |
+---------+-------------+-----------+------------+-----------+-----------------+
|       1 | Иванов      | Иван      | Петрович   |         1 | NULL            |
|       2 | Иволгин     | Пётр      | Николаевич |         1 | NULL            |
|       3 | Петров      | Павел     | Сергеевич  |         1 | NULL            |
|       4 | Сидоров     | Александр | Сергеевич  |         2 | NULL            |
|       5 | Павлов      | Дмитрий   | Михайлович |         2 | NULL            |
|       6 | Немцов      | Борис     | Петрович   |         3 | NULL            |
|       7 | Харитонова  | Алла      | Николаевна |         3 | NULL            |
|       8 | Парамонова  | Татьяна   | Викторовна |         3 | NULL            |
|       9 | Иванов      | Сергей    | Сергеевич  |         3 | NULL            |
|      10 | Александров | Игорь     | Николаевич |         3 | NULL            |
+---------+-------------+-----------+------------+-----------+-----------------+
10 rows in set (0.03 sec)
mysql> _
```

Рис. 3.1

Выборочное извлечение полей из таблицы

Формат оператора select:

select <список полей> from <имя таблицы>

На рис. 3.2 представлен пример извлечения информации из полей Fam и gr_nomber таблицы student.

```
MySQL Command Line Client
mysql> select fam,gr_nomber from student;
+-------------+-----------+
| fam         | gr_nomber |
+-------------+-----------+
| Иванов      |         1 |
| Иволгин     |         1 |
| Петров      |         1 |
| Сидоров     |         2 |
| Павлов      |         2 |
| Немцов      |         3 |
| Харитонова  |         3 |
| Парамонова  |         3 |
| Иванов      |         3 |
| Александров |         3 |
+-------------+-----------+
10 rows in set (0.00 sec)
```

Рис. 3.2

Имена полей запроса могут отличаться от имён полей таблицы – источника. На рис. 3.3 представлен пример такого переименования полей запроса.

```
MySQL Command Line Client
mysql> select fam as 'Фамилия студента',gr_nomber as '№ уч. гр.' from student;
+------------------+-----------+
| Фамилия студента | № уч. гр. |
+------------------+-----------+
| Иванов           |         1 |
| Иволгин          |         1 |
| Петров           |         1 |
| Сидоров          |         2 |
| Павлов           |         2 |
| Немцов           |         3 |
| Харитонова       |         3 |
| Парамонова       |         3 |
| Иванов           |         3 |
| Александров      |         3 |
+------------------+-----------+
10 rows in set (0.01 sec)

mysql>
```

Рис. 3.3

Кроме того, в списке полей допустимы и произвольные (например, строковые) выражения (рис. 3.4).

```
MySQL Command Line Client
mysql> select concat('студент  ',fam) as 'Фамилия',
    -> concat('№  ',gr_nomber) as 'уч. гр.' from student;
+----------------------+---------+
| Фамилия              | уч. гр. |
+----------------------+---------+
| студент  Иванов      | №  1    |
| студент  Иволгин     | №  1    |
| студент  Петров      | №  1    |
| студент  Сидоров     | №  2    |
| студент  Павлов      | №  2    |
| студент  Немцов      | №  3    |
| студент  Харитонова  | №  3    |
| студент  Парамонова  | №  3    |
| студент  Иванов      | №  3    |
| студент  Александров | №  3    |
+----------------------+---------+
10 rows in set (0.00 sec)

mysql> _
```

Рис. 3.4

Сортировка записей в операторе select

Формат:

select <список полей> **from** <имя таблицы> **order by <способ сортировки>**

Способ сортировки представляет собой список полей (или выражений), разделенных запятыми после каждого из которых может стоять одно из слов: **ASC** — сортировка по данному полю осуществляется по возрастанию, **DESC** — по убыванию. Если после имени поля не стоит ни одно из этих слов, то подразумевается сортировка по возрастанию. Сортировка строк осуществляется сначала по первому полю из списка, если две строки имеют равные значения первого поля, то — по второму и т.д.

На рис. 3.5 представлен пример вывода из таблицы student фамилий студентов, отсортированных по убыванию и кодов студентов, отсортированных по возрастанию.

```
MySQL Command Line Client
mysql> select kodstud,fam from student order by fam desc,kodstud asc;
+---------+-------------+
| kodstud | fam         |
+---------+-------------+
|       7 | Харитонова  |
|       4 | Сидоров     |
|       3 | Петров      |
|       5 | Павлов      |
|       8 | Парамонова  |
|       6 | Немцов      |
|       2 | Иволгин     |
|       1 | Иванов      |
|       9 | Иванов      |
|      10 | Александров |
+---------+-------------+
10 rows in set (0.02 sec)

mysql>
```

Рис. 3.5

На рис. 3.6 представлен пример вывода из таблицы student содержимого всех полей, при этом отчества студентов отсортированы по убыванию.

```
MySQL Command Line Client
mysql> select * from student order by otch desc;
+---------+-------------+-----------+------------+-----------+-----------------+
| kodStud | fam         | imya      | otch       | gr_nomber | change_address  |
+---------+-------------+-----------+------------+-----------+-----------------+
|       9 | Иванов      | Сергей    | Сергеевич  |         3 | NULL            |
|       3 | Петров      | Павел     | Сергеевич  |         1 | NULL            |
|       4 | Сидоров     | Александр | Сергеевич  |         2 | NULL            |
|       1 | Иванов      | Иван      | Петрович   |         1 | NULL            |
|       6 | Немцов      | Борис     | Петрович   |         3 | NULL            |
|       7 | Харитонова  | Алла      | Николаевна |         3 | NULL            |
|       2 | Иволгин     | Пётр      | Николаевич |         1 | NULL            |
|      10 | Александров | Игорь     | Николаевич |         3 | NULL            |
|       5 | Павлов      | Дмитрий   | Михайлович |         2 | NULL            |
|       8 | Парамонова  | Татьяна   | Викторовна |         3 | NULL            |
+---------+-------------+-----------+------------+-----------+-----------------+
10 rows in set (0.00 sec)

mysql>
```

Рис. 3.6

Выборочное извлечение записей оператором select

Исключение повторяющихся строк

В таблице с первичным ключом таких записей быть не может, но если из таблицы выбираются не все поля, а только их часть, то они могут появиться. Например, в таблице student имеется 2 студента по фамилии Иванов. Для исключения повторов после оператора **select** ставится солужебное слово **distinct**, которое ставится после select. На рис. 3.7 показано решение задачи вывода упорядоченного по возрастанию списка учебных групп, имеющихся в таблице student.

```
MySQL Command Line Client
mysql> select distinct gr_nomber from student order by gr_nomber;
+-----------+
| gr_nomber |
+-----------+
|         1 |
|         2 |
|         3 |
+-----------+
3 rows in set (0.00 sec)

mysql>
```

Рис. 3.7

Ограничение количества строк, возвращенных командой SELECT.

Выражение LIMIT принимает один или два числовых аргумента. Эти аргументы должны быть целочисленными константами. Если заданы два аргумента, то первый указывает на начало первой возвращаемой строки, а второй задает максимальное количество возвращаемых строк. При этом смещение начальной строки равно 0 (не 1):

mysql> SELECT * FROM table LIMIT 5,10; возвращает строки 6-15.

Если задан один аргумент, то он показывает максимальное количество возвращаемых строк: mysql> SELECT * FROM table LIMIT 5; возвращает первых 5 строк.

Другими словами, LIMIT n эквивалентно LIMIT 0,n.

Инструкция where

Формат: select … from … where <условие> [order by …]

Условие может принимать одно из трех значений: TRUE (истина), FALSE (ложь) и UNKNOWN (неизвестно). В результате выполнения такого оператора выбираются в точности те строки, для которых условие истинно, строки для которых условие ложно или неизвестно отбрасываются. Условие в простейшем случае представляет собой сравнения полей с константами и другими полями. Простейшие условия можно соединять логическими операциями and, or и not по правилам трехзначной логики:

And	T	U	F
T	T	U	F
U	U	U	F
F	F	F	F

Or	T	U	F
T	T	T	T
U	T	U	U
F	T	U	F

Not	
T	F
U	U
F	T

Использование операторов сравнения

В качестве сравнений используются равенство (=), неравенство (<>), отношения порядка (<, >, <=, >=). Если одно из сравниваемых значений есть NULL, то результат сравнения — UNKNOWN (это же относится и к любым другим условиям). В противном случае результат всегда является истинным или ложным. На рис. 3.8 представлен результат запроса по полям KodStud (>2) и Otch (<>'Сергеевич').

```
MySQL Command Line Client
mysql> select * from student where kodStud>2 and otch<>'Сергеевич';
+---------+-------------+---------+------------+-----------+----------------+
| kodStud | fam         | imya    | otch       | gr_nomber | change_address |
+---------+-------------+---------+------------+-----------+----------------+
|       5 | Павлов      | Дмитрий | Михайлович |         2 | NULL           |
|       6 | Немцов      | Борис   | Петрович   |         3 | NULL           |
|       7 | Харитонова  | Алла    | Николаевна |         3 | NULL           |
|       8 | Парамонова  | Татьяна | Викторовна |         3 | NULL           |
|      10 | Александров | Игорь   | Николаевич |         3 | NULL           |
+---------+-------------+---------+------------+-----------+----------------+
5 rows in set (0.06 sec)

mysql>
```

Рис. 3.8

Использование BETWEEN

Условие принадлежности значения поля некоторому отрезку значений**:**

<выражение> between <выражение1> and <выражение2>

Условия принадлежности могут объединяться логическими условиями, например, как показано на рис. 3.9.

```
MySQL Command Line Client
    -> where kodStud between 2 and 5
    ->    or kodStud between 8 and 10 order by otch;
+-----------+
| Imya      |
+-----------+
| Татьяна   |
| Дмитрий   |
| Пётр      |
| Игорь     |
| Павел     |
| Александр |
| Сергей    |
+-----------+
7 rows in set (0.02 sec)

mysql>
```

Рис. 3.9

Использование IN

Для проверки на равенство одному из нескольких значений используется in:

<выражение> in (<список выражений>)

Такое условие истинно, если значение выражения равняется значению одного из элементов списка. На рис. 3.10 представлен пример выбора из табл. Student фамилий студентов только 2 и 3 групп.

```
MySQL Command Line Client
mysql> select fam from student where gr_nomber in (2,3);
+-------------+
| fam         |
+-------------+
| Сидоров     |
| Павлов      |
| Немцов      |
| Харитонова  |
| Парамонова  |
| Иванов      |
| Александров |
+-------------+
7 rows in set (0.00 sec)

mysql> _
```

Рис. 3.10

Использование LIKE

Обычная форма "имя_столбца LIKE текстовая_константа (шаблон)" для столбца текстового типа позволяет отыскать все значения указанного столбца, соответствующие образцу, заданному "текстовой_константой". Символы этой константы интерпретируются следующим образом:

- символ _ (подчеркивание) – заменяет любой одиночный символ,
- символ % (процент) – заменяет любую последовательность из N символов (где N может быть нулем).

На рис. 3.11 представлен пример выбора из табл. Prep фамилий и должностей преподавателей, чьи фамилии начинаются на букву 'С'.

```
MySQL Command Line Client
mysql> select famPrep,dolzh from prep where famPrep like 'С%';
+----------+-----------+
| famPrep  | dolzh     |
+----------+-----------+
| Ситников | доцент    |
| Столыпин | профессор |
+----------+-----------+
2 rows in set (0.00 sec)

mysql>
```

Рис. 3.11

На рис. 3.12 представлен пример выбора из табл. Stud кодов и фамилий студентов, у которых вторая буква имени 'а'.

```
MySQL Command Line Client
mysql> select kodStud,fam from student where imya like '_а%';
+---------+------------+
| kodStud | fam        |
+---------+------------+
|       3 | Петров     |
|       8 | Парамонова |
+---------+------------+
2 rows in set (0.05 sec)

mysql>
```

Рис. 3.12

Группировка данных и агрегатные функции

Один из основных способов анализа данных — использование инструкции group by в операторе select:

select … from …
[where …]
group by <список грушшировки>
[order by . . .]

Список группировки является списком полей, разделенных запятыми. В результате все строки объединяются в несколько групп таким образом, что каждую группу образуют строки, у которых значения всех столбцов их списка группировки совпадают. Затем из каждой группы формируется одна строка. Для того, чтобы из каждой группы выбрать одно значение для какого либо поля или сгенерировать новое значение, используются агрегатные функции:

- count (*) — подсчет количества строк в группе;
- max —выбор максимального значения;
- min — выбор минимального значения;
- avg — вычисление среднего арифметического в группе;
- sum — вычисление суммы в группе.

Последние две функции могут использоваться только для полей числового типа. Поля, формируемые с помощью агрегатных функций, не имеют имени, поэтому обычно им требуется явно присваивать имя при помощи **as.**

Отсюда следует, что в списке выбираемых полей после select нельзя использовать поля, не входящие в список группировки. Например, если в списке выбираемых полей из табл. student указать gr_nomber и kodStud, то ввиду уникальности поля kodStud, совпадающих записей по этому полю быть не может, а следовательно результат запроса будет неверным (на рис 3.13 в поле kodStud присутствуют только коды студентов, имеющих наименьшие значения этого поля в каждой из групп). Пример корректного использования инструкции group by представлен на рис. 3.14. На рис. 3.15 представлен пример определения min и max кодов студентов из табл. Student в каждой группе.

```
MySQL Command Line Client
mysql> select gr_nomber,kodStud,count(*) as 'Кол. студ' from student group by gr
_nomber;
+-----------+---------+-----------+
| gr_nomber | kodStud | Кол. студ |
+-----------+---------+-----------+
|         1 |       1 |         3 |
|         2 |       4 |         2 |
|         3 |       6 |         5 |
+-----------+---------+-----------+
3 rows in set (0.00 sec)

mysql>
```

Рис 3.13

```
MySQL Command Line Client
mysql> select gr_nomber as'№ уч. гр.',count(*) as 'Кол. студ'
    -> from student group by gr_nomber;
+-----------+-----------+
| № уч. гр. | Кол. студ |
+-----------+-----------+
|         1 |         3 |
|         2 |         2 |
|         3 |         5 |
+-----------+-----------+
3 rows in set (0.00 sec)

mysql>
```

Рис. 3.14

```
MySQL Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 5.0.67-community-nt MySQL Community Edition (GPL)

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> use dbstud_lessons
Database changed
mysql> select gr_nomber,min(kodStud) as 'Миним. код студента',
    -> max(kodStud) as 'Максим. код студента'from student group by gr_nomber;
+-----------+----------------------+----------------------+
| gr_nomber | Миним. код студента | Максим. код студента |
+-----------+----------------------+----------------------+
|         1 |                    1 |                    3 |
|         2 |                    4 |                    5 |
|         3 |                    6 |                   10 |
+-----------+----------------------+----------------------+
3 rows in set (0.05 sec)

mysql>
```

Рис. 3.15

Совместное использование инструкций group by и where

В этом случае сначала осуществляется отбор строк при помощи **where,** а затем — группировка. На рис. 3.16 представлен пример вычисления среднего балла по дисциплинам, преподаваемых преподавателем № 3. На этом примере видно, что в инструкции **where** можно использовать имена тех столбцов, по которым группировка не проводиться (примере — kodPrep**).** Это связано с тем, что фильтрация строк в **where** осуществляется до группировки.

```
MySQL Command Line Client
mysql> select koddis,avg(ball) as 'Сред.балл' from zanstud
    -> where kodprep=3 group by koddis;
+--------+-----------+
| koddis | Сред.балл |
+--------+-----------+
|      3 |    4.1429 |
+--------+-----------+
1 row in set (0.01 sec)

mysql> _
```

Рис. 3.16

Агрегатные функции без использования инструкции group by

В этом случае считается, что все строки образуют одну группу. На рис. 3.17 представлен пример определения среднего балла по всем студентам и дисциплинам, а на рис. 3.18 – только по дисциплине Информатика.

```
MySQL Command Line Client
mysql> select avg(ball) as 'Ср. по всем студ. и дисц.' from zanStud;
+---------------------------+
| Ср. по всем студ. и дисц. |
+---------------------------+
|                    3.9167 |
+---------------------------+
1 row in set (0.00 sec)

mysql>
```

Рис. 3.17

```
MySQL Command Line Client
mysql> select avg(ball) as 'Ср. по дисц. Информатика' from zanStud
    -> where kodDis=1;
+--------------------------+
| Ср. по дисц. Информатика |
+--------------------------+
|                   3.6667 |
+--------------------------+
1 row in set (0.00 sec)

mysql>
```

Рис. 3.18

Отбор записей инструкцией having

Для того, чтобы в результирующей таблице оставить только нужные записи (исключить ненужные строки) после группировки используется инструкция **having:**

select … from … [where …] group by . . . having <условие присутствия>

Поскольку **having** выполняется после формирования групп, то в условии после **having** можно использовать агрегатные функции, но нельзя использовать поля, не вошедшие в список группировки. На рис. 3.19 представлен пример определения средних баллов по всем дисциплинам кроме Информатики (код этой дисциплины в табл. zanStud = 1), выведенных в порядке убывания среднего балла.

```
MySQL Command Line Client
mysql> select koddis as 'Код дисцип.',avg(ball) as 'Ср. балл кроме Информатики'
    -> from zanStud group by kodDis having kodDis<>1
    -> order by avg(ball) desc;
+-------------+----------------------------+
| Код дисцип. | Ср. балл кроме Информатики |
+-------------+----------------------------+
|           3 |                     4.1429 |
|           2 |                     3.5000 |
+-------------+----------------------------+
2 rows in set (0.02 sec)

mysql>
```

Рис. 3.19

На рис. 3.20 представлен пример выдачи средних баллов по всем дисциплинам в случае, если он выше 3,6.

```
MySQL Command Line Client
mysql> select koddis as 'Код дисцип.',avg(ball) as 'Ср. балл'
    -> from zanStud group by kodDis having avg(ball)>3.6;
+-------------+----------+
| Код дисцип. | Ср. балл |
+-------------+----------+
|           1 |   3.6667 |
|           3 |   4.1429 |
+-------------+----------+
2 rows in set (0.02 sec)

mysql>
```

Рис. 3.20

Соединение таблиц

Это операция, при которой к каждой строке одной таблицы приписываются все строки другой, удовлетворяющие некоторым условиям. Результатом соединения является новая таблица, содержащая все столбцы первой таблицы и все столбцы второй.

Соединения делят на внутренние и внешние. При внутреннем соединении все данные получаемой таблицы берутся из исходных таблиц. При внешних соединениях часть данных новой таблицы не соответствует данным исходных.

Внутреннее соединение

Самый простой способ внутреннего соединения — декартово произведение исходных таблиц. Декартово произведение записывается двумя способами: соединяя имена таблиц словами **cross join** или запятой:

from <имя таблицы> cross join <имя таблицы> или

from <имя таблицы>, <имя таблицы>

Пример декартова произведения таблиц Student и zanStud (Оба оператора выдадут один и тот же результат):

select * from student cross join zanStud

или

select * from student, zanStud

Простое декартово произведение используется редко. Чаще требуется только его часть. Например, нужно чтобы каждая строка таблицы **student** соединялась лишь с теми строками таблицы **zanStud**, у которых поля, содержащие код студента, совпадают. Этого можно достичь двумя способами. Первый — в один из предыдущих операторов вставить инструкцию **where** с соответствующим условием:

select …from <имя таблицы>, <имя таблицы> where <условие соединения>

Например, на рис. 3.21 представлен результат выполнения команды (при совпадении имён полей различных таблиц приходится предварять их квалификаторами в виде имён таблиц):

select * from student, zanstud where student.kodstud = zanstud.kodstud;

Второй способ — использовать соединение при помощи **inner join . . . on:**

select …from <имя таблицы> inner join <имя таблицы> on <условие соединения>

Для рассмотренного примера тот же результат достигается командой:

select * from student inner join zanstud on student.kodstud = zanstud.kodstud;

```
MySQL Command Line Client
mysql> select * from student,zanstud where student.kodstud=zanstud.kodstud;
+---------+------------+-----------+------------+-----------+-----------------+--
----------+---------+--------+---------+--------------+------+------+
| kodStud | fam        | imya      | otch       | gr_nomber | change_address | k
odZanStud | kodStud | koddis | kodprep | ball_kursRab | test | ball |
+---------+------------+-----------+------------+-----------+-----------------+--
----------+---------+--------+---------+--------------+------+------+
|       1 | Иванов     | Иван      | Петрович   |         1 | NULL           |
        1 |       1 |      1 |       1 |            3 |      |    3 |
|       2 | Иволгин    | Пётр      | Николаевич |         1 | NULL           |
        2 |       2 |      1 |       1 |         NULL |      |    5 |
|       3 | Петров     | Павел     | Сергеевич  |         1 | NULL           |
        3 |       3 |      1 |       1 |         NULL | NULL |    3 |
|       4 | Сидоров    | Александр | Сергеевич  |         2 | NULL           |
        4 |       4 |      2 |       2 |            3 | NULL |    3 |
|       5 | Павлов     | Дмитрий   | Михайлович |         2 | NULL           |
        5 |       5 |      2 |       2 |         NULL | NULL |    4 |
|       6 | Немцов     | Борис     | Петрович   |         3 | NULL           |
        6 |       6 |      3 |       3 |         NULL | NULL |    3 |
|       7 | Харитонова | Алла      | Николаевна |         3 | NULL           |
        7 |       7 |      3 |       3 |            3 | NULL |    5 |
|       8 | Парамонова | Татьяна   | Викторовна |         3 | NULL           |
        8 |       8 |      3 |       3 |         NULL | NULL |    4 |
|       9 | Иванов     | Сергей    | Сергеевич  |         3 | NULL           |
        9 |       9 |      3 |       3 |         NULL | NULL |    4 |
|       1 | Иванов     | Иван      | Петрович   |         1 | NULL           |
       10 |       1 |      3 |       3 |         NULL | NULL |    4 |
|       2 | Иволгин    | Пётр      | Николаевич |         1 | NULL           |
       11 |       2 |      3 |       3 |         NULL | NULL |    5 |
|       3 | Петров     | Павел     | Сергеевич  |         1 | NULL           |
       12 |       3 |      3 |       3 |         NULL | NULL |    4 |
+---------+------------+-----------+------------+-----------+-----------------+--
----------+---------+--------+---------+--------------+------+------+
12 rows in set (0.00 sec)

mysql>
```

Рис. 3.21

Из рисунка видно, что таблица получилась достаточно громоздкой. С одной стороны она содержит много столбцов, с другой — эти столбцы повторяются. Если для каждого студента нужно вывести только оценки по каждой дисциплине, то для выборки целесообразно использовать запрос, представленный на рис. 3.22. В случае, если необходимо вывести все поля (например, таблицы student), то запрос будет выглядеть следующим образом:

select student.*, kodDis, ball from student, zanstud
where student.kodStud = zanstud. kodStud;

```
MySQL Command Line Client
mysql> select student.kodStud,fam,koddis,ball from student,zanStud
    -> where student.kodstud=zanstud.kodstud;
+---------+------------+--------+------+
| kodStud | fam        | koddis | ball |
+---------+------------+--------+------+
|       1 | Иванов     |      1 |    3 |
|       2 | Иволгин    |      1 |    5 |
|       3 | Петров     |      1 |    3 |
|       4 | Сидоров    |      2 |    3 |
|       5 | Павлов     |      2 |    4 |
|       6 | Немцов     |      3 |    3 |
|       7 | Харитонова |      3 |    5 |
|       8 | Парамонова |      3 |    4 |
|       9 | Иванов     |      3 |    4 |
|       1 | Иванов     |      3 |    4 |
|       2 | Иволгин    |      3 |    5 |
|       3 | Петров     |      3 |    4 |
+---------+------------+--------+------+
12 rows in set (0.00 sec)

mysql>
```

Рис. 3.22

Соединение более двух таблиц осуществляется последовательно:

<имя таблицы> inner join <имя таблицы> on <условие>
inner join <имя таблицы> on <условие>

На рис. 3.23 представлен результат соединения всех 4 таблиц учебной БД (студенты, студенты и занятия, дисциплины, преподаватели).

```
MySQL Command Line Client
mysql> select student.kodstud,fam,dis.dis_name,zanstud.ball,prep.famPrep
    -> from student inner join zanstud on student.kodstud=zanstud.kodstud
    -> inner join dis on zanstud.koddis=dis.koddis
    -> inner join prep on dis.kodprep=prep.kodprep;
+---------+------------+----------------------+------+----------+
| kodstud | fam        | dis_name             | ball | famPrep  |
+---------+------------+----------------------+------+----------+
|       1 | Иванов     | Информатика          |    3 | Ситников |
|       2 | Иволгин    | Информатика          |    5 | Ситников |
|       3 | Петров     | Информатика          |    3 | Ситников |
|       4 | Сидоров    | Системный анализ     |    3 | Столыпин |
|       5 | Павлов     | Системный анализ     |    4 | Столыпин |
|       6 | Немцов     | Матем. моделирование |    3 | Крапивин |
|       7 | Харитонова | Матем. моделирование |    5 | Крапивин |
|       8 | Парамонова | Матем. моделирование |    4 | Крапивин |
|       9 | Иванов     | Матем. моделирование |    4 | Крапивин |
|       1 | Иванов     | Матем. моделирование |    4 | Крапивин |
|       2 | Иволгин    | Матем. моделирование |    5 | Крапивин |
|       3 | Петров     | Матем. моделирование |    4 | Крапивин |
+---------+------------+----------------------+------+----------+
12 rows in set (0.02 sec)

mysql>
```

Рис. 3.23

На рис. 3.24 представлен этот же результат, полученный с помощью второго варианта кода.

```
MySQL Command Line Client
mysql> select student.kodstud,fam,dis.dis_name,zanstud.ball,prep.famPrep
    -> from student,zanStud,dis,prep
    -> where student.kodstud=zanstud.kodstud and zanstud.kodDis=dis.kodDis
    -> and dis.kodPrep=prep.kodPrep;
+---------+------------+----------------------+------+----------+
| kodstud | fam        | dis_name             | ball | famPrep  |
+---------+------------+----------------------+------+----------+
|       1 | Иванов     | Информатика          |    3 | Ситников |
|       2 | Иволгин    | Информатика          |    5 | Ситников |
|       3 | Петров     | Информатика          |    3 | Ситников |
|       4 | Сидоров    | Системный анализ     |    3 | Столыпин |
|       5 | Павлов     | Системный анализ     |    4 | Столыпин |
|       6 | Немцов     | Матем. моделирование |    3 | Крапивин |
|       7 | Харитонова | Матем. моделирование |    5 | Крапивин |
|       8 | Парамонова | Матем. моделирование |    4 | Крапивин |
|       9 | Иванов     | Матем. моделирование |    4 | Крапивин |
|       1 | Иванов     | Матем. моделирование |    4 | Крапивин |
|       2 | Иволгин    | Матем. моделирование |    5 | Крапивин |
|       3 | Петров     | Матем. моделирование |    4 | Крапивин |
+---------+------------+----------------------+------+----------+
12 rows in set (0.00 sec)

mysql> _
```

Рис. 3.24

Внешнее соединение

Внешнее соединение предназначено для сохранения всех записей одной из таблиц или обеих сразу. Синтаксис конструкций внешних соединений:

- левое — **<имя таблицы> left outer join <имя таблицы> on <условие>**
- правое — **<имя таблицы> right outer join <имя таблицы> on <условие>**
- полное — **<имя таблицы> full outer join <имя таблицы> on <условие>**

В первом случае все строки первой таблицы попадут в результат соединения, во втором случае — все строки второй таблицы, в третьем — все строки и той и другой будут присутствовать.

Левое соединение. Если для какой-то строки первой таблицы нашлась хоть одна строка во второй, которая удовлетворяет условию соединения, то внешнее соединение ведет себя для этой строки так же, как внутреннее. Если же для строки первой таблицы пары из второй не нашлось, то такая строка тоже включается в результат, но все поля второй таблицы для этой строки принимают значение NULL, означающее отсутствие данных.

На рис. 3.25 представлен пример определения ср. балла экзаменов по преподавателям.

```
MySQL Command Line Client
mysql> select famprep,avg(ball) from prep left outer join
    -> zanstud on prep.kodprep=zanstud.kodprep group by famprep;
+----------+-----------+
| famprep  | avg(ball) |
+----------+-----------+
| Крапивин |    4.1429 |
| Полушкин |      NULL |
| Столыпин |    3.5000 |
| Ситников |    3.6667 |
+----------+-----------+
4 rows in set (0.02 sec)

mysql>
```

Рис. 3.25

Любые действия с NULL в результате дают NULL, в частности, при нахождении среднего арифметического avg результатом будет NULL, если NULL встречается хотя бы в одной записи группы. Для проверки, имеет ли некоторое выражение значение **NULL,** используется предикат **is null.** На рис. 3.26 представлен пример вывода

наименований дисциплин и фамилий преподавателей этих дисциплин, зачёты по которым не внесены в БД.

```
MySQL Command Line Client
mysql> select dis_name,famprep from dis left outer join
    -> (prep inner join zanstud on prep.kodprep=zanstud.kodprep)
    -> on dis.koddis=zanstud.koddis
    -> where test is null;
+----------------------+----------+
| dis_name             | famprep  |
+----------------------+----------+
| Информатика          | Ситников |
| Информатика          | Ситников |
| Системный анализ     | Столыпин |
| Системный анализ     | Столыпин |
| Матем. моделирование | Крапивин |
| Матем. моделирование | Крапивин |
| Матем. моделирование | Крапивин |
| Матем. моделирование | Крапивин |
| Матем. моделирование | Крапивин |
| Матем. моделирование | Крапивин |
| Матем. моделирование | Крапивин |
| Экон. теория         | NULL     |
+----------------------+----------+
12 rows in set (0.00 sec)

mysql>
```

Рис. 3.26

Правое внешнее соединение отличается от левого только тем, что в результат включаются все строки из правой таблицы, а значение NULL записывается в столбцы левой таблицы, если соответствующие строки в ней отсутствуют.

Полное внешнее соединение сохраняет все строки и той и другой таблицы.

Подзапросы

Вложенный подзапрос - это подзапрос, заключенный в круглые скобки и вложенный в WHERE (HAVING) фразу предложения SELECT или других предложений, использующих WHERE фразу. Вложенный подзапрос может содержать в своей WHERE (HAVING) фразе другой вложенный подзапрос и т.д. Нетрудно догадаться, что вложенный подзапрос создан для того, чтобы при отборе строк таблицы, сформированной основным запросом, можно было использовать данные из других таблиц (например, при отборе блюд для меню использовать данные о наличии продуктов в кладовой пансионата).

Существуют простые и коррелированные вложенные подзапросы. Они включаются в WHERE (HAVING) фразу с помощью условий IN, EXISTS или одного из условий сравнения (= | <> | < | <= | > | > =). Простые вложенные подзапросы обрабатываютя системой "снизу вверх". Первым обрабатывается вложенный подзапрос самого нижнего уровня. Множество значений, полученное в результате его выполнения, используется при реализации подзапроса более высокого уровня и т.д. Запросы с коррелированными вложенными подзапросами обрабатываются системой в обратном порядке. Сначала выбирается первая строка рабочей таблицы, сформированной основным запросом, и из нее выбираются значения тех столбцов, которые используются во вложенном подзапросе (вложенных подзапросах). Если эти значения удовлетворяют условиям вложенного подзапроса, то выбранная строка включается в результат. Затем выбирается вторая строка и т.д., пока в результат не будут включены все строки, удовлетворяющие вложенному подзапросу (последовательности вложенных подзапросов).

Следует отметить, что SQL обладает большой избыточностью в том смысле, что он часто предоставляет несколько различных способов формулировки одного и того же запроса. И несмотря на то, что часть из них успешнее реализуется с помощью соединений, все же будут приведены их варианты с использованием вложенных подзапросов. Это связано с необходимостью детального знакомства с созданием и принципом выполнения

вложенных подзапросов, так как существует немало задач (особенно на удаление и изменение данных), которые не могут быть реализованы другим способом. Кроме того, разные формулировки одного и того же запроса требуют для своего выполнения различных ресурсов памяти и могут значительно отличаться по времени реализации в разных СУБД.

Самой общей формой использования подзапросов является использование его в качестве условия в инструкциях **where** или **having:**

|exists(<SQL-3апрос>)

Это условие будет истинным, если таблица, сгенерированная в результате выполнения SQL-запроса в скобках, содержит хотя бы одну строку, в противном случае условие ложно. Частные случаи использования подзапросов — сравнения:

<выражение> <знак сравнения> any (<SQL-3апрос>)

и **<выражение> <знак сравнения> all (<SQL-3апрос>)**

В обоих случаях SQL-запрос должен возвращать таблицу с одним столбцом, значения которого можно сравнивать с выражением. Первое условие будет истинным, если сравнение выполняется хотя бы для одной строки сгенерированной таблицы, второе — если условие выполняется для всех строк. В частности, если вложенный запрос вернет пустую таблицу, то первое условие ложно, а второе истинно. Если вложенный запрос возвращает ровно одну строку, то оба условия эквивалентны.

На рис. 3.27 представлен поиск студентов, имеющих самые высокие оценки.

```
MySQL Command Line Client
mysql> select student.kodStud,fam,zanstud.koddis, ball
    -> from student inner join zanStud on student.kodstud=zanstud.kodstud
    -> where ball=all(select max(ball) from zanstud);
+---------+------------+--------+------+
| kodStud | fam        | koddis | ball |
+---------+------------+--------+------+
|       2 | Иволгин    |      1 |    5 |
|       7 | Харитонова |      3 |    5 |
|       2 | Иволгин    |      3 |    5 |
+---------+------------+--------+------+
3 rows in set (0.00 sec)

mysql>
```

Рис. 3.27

На рис. 3.28 представлен изменённый запрос, ищущий студентов, получивших самый высокий балл по каждому из предметов:

```
MySQL Command Line Client
mysql> select student.kodStud,fam,zanstud.koddis, ball
    -> from student inner join zanStud on student.kodstud=zanstud.kodstud
    -> where ball=all(select max(ball) from zanstud
    ->        b where zanstud.koddis=b.koddis);
+---------+------------+--------+------+
| kodStud | fam        | koddis | ball |
+---------+------------+--------+------+
|       2 | Иволгин    |      1 |    5 |
|       5 | Павлов     |      2 |    4 |
|       7 | Харитонова |      3 |    5 |
|       2 | Иволгин    |      3 |    5 |
+---------+------------+--------+------+
4 rows in set (0.00 sec)

mysql> _
```

Рис. 3.28

Здесь при выполнении подзапроса используется поле kodDis из таблицы zanStud основного запроса. Такие подзапросы называют связанными. В связи с использованием таблицы zanStud и во внешнем запросе, и в подзапросе возникает необходимость переименования таблицы вложенного запроса, чтобы отличать ее от таблицы основного

запроса, и необходимость явного указания таблиц для поля kodDis. На рис. 3.29 представлен запрос, выводящий все оценки по каждому предмету, кроме самых низких.

```
MySQL Command Line Client
    -> where ball>any(select min(ball) from zanstud
    ->         b where zanstud.koddis=b.koddis);
ERROR 1054 (42S22): Unknown column 'dis_name' in 'field list'
mysql> select student.kodStud,fam,zanstud.koddis, ball
    -> from student inner join zanStud on student.kodstud=zanstud.kodstud
    -> where ball>any(select min(ball) from zanstud
    ->         b where zanstud.koddis=b.koddis);
+---------+------------+--------+------+
| kodStud | fam        | koddis | ball |
+---------+------------+--------+------+
|       2 | Иволгин    |      1 |    5 |
|       5 | Павлов     |      2 |    4 |
|       7 | Харитонова |      3 |    5 |
|       8 | Парамонова |      3 |    4 |
|       9 | Иванов     |      3 |    4 |
|       1 | Иванов     |      3 |    4 |
|       2 | Иволгин    |      3 |    5 |
|       3 | Петров     |      3 |    4 |
+---------+------------+--------+------+
```

Рис. 3.29

Квантор EXISTS (существует) - понятие, заимствованное из формальной логики. В языке SQL предикат с квантором существования представляется выражением EXISTS (SELECT * FROM ...). Такое выражение считается истинным только тогда, когда результат вычисления "SELECT * FROM ..." является непустым множеством, т.е. когда существует какая-либо запись в таблице, указанной во фразе FROM подзапроса, которая удовлетворяет условию WHERE подзапроса. (Практически этот подзапрос всегда будет коррелированным множеством).

Каждый из предыдущих запросов можно записать при помощи exists. На рис. 3.30 представлен поиск студентов, имеющих самые высокие оценки (см. рис. 3.27). На рис. 3.31 представлен запрос, ищущий студентов, получивших самый высокий балл по каждому из предметов (см. рис. 3.28).

```
MySQL Command Line Client
mysql> select student.kodstud,fam,zanstud.koddis,ball
    ->     from student inner join zanstud
    -> on student.kodstud=zanstud.kodstud where not exists (
    -> select * from zanstud b where b.ball > zanstud.ball);
+---------+------------+--------+------+
| kodstud | fam        | koddis | ball |
+---------+------------+--------+------+
|       2 | Иволгин    |      1 |    5 |
|       7 | Харитонова |      3 |    5 |
|       2 | Иволгин    |      3 |    5 |
+---------+------------+--------+------+
3 rows in set (0.00 sec)

mysql>
```

Рис. 3.30

```
MySQL Command Line Client
mysql> select student.kodstud,fam,zanstud.koddis,ball
    ->     from student inner join zanstud
    -> on student.kodstud=zanstud.kodstud where not exists (
    -> select * from zanstud b where b.ball > zanstud.ball and
    -> zanstud.koddis=b.koddis);
+---------+------------+--------+------+
| kodstud | fam        | koddis | ball |
+---------+------------+--------+------+
|       2 | Иволгин    |      1 |    5 |
|       5 | Павлов     |      2 |    4 |
|       7 | Харитонова |      3 |    5 |
|       2 | Иволгин    |      3 |    5 |
+---------+------------+--------+------+
4 rows in set (0.00 sec)

mysql> _
```

Рис. 3.31

Еще одна форма использования подзапросов — условие:

<выражение> in (<SQL-3апрос>)

Здесь, как и в сравнениях, **any** и **all** SQL-запрос должен возвращать таблицу с одним столбцом, тип которого совместим с выражением. Условие считается истинным, если этот столбец содержит значение выражения. То есть, приведенное условие эквивалентно такому:

<выражение> = any(<SQL-3апрос>)

Например, чтобы выбрать студентов, которые ни за один предмет не получили меньше 4 баллов, можно воспользоваться запросом, представленным на рис. 3.32.

```
MySQL Command Line Client
mysql> select student.kodstud,fam
    ->   from student where student.kodstud not in
    ->       (select zanstud.kodstud from zanstud where ball<4);
+---------+-------------+
| kodstud | fam         |
+---------+-------------+
|       2 | Иволгин     |
|       5 | Павлов      |
|       7 | Харитонова  |
|       8 | Парамонова  |
|       9 | Иванов      |
|      10 | Александров |
+---------+-------------+
6 rows in set (0.01 sec)

mysql> _
```

Рис. 3.32

Объединение таблиц

Предположим, нужно напечатать список пригласительных билетов на вечер, куда приглашаются и преподаватели, и студенты. Сложность создания такого списка в том, что фамилии преподавателей и студентов хранятся в разных таблицах, а в результате они должны составить один столбец таблицы приглашенных. Для выполнения этой задачи лучше всего использовать инструкцию объединения **union**.

Все операторы **select**, участвующие в инструкции union, должны генерировать таблицы с одинаковым количеством столбцов и одинаковыми типами столбцов. При этом следует иметь ввиду, что инструкция **union** неявно применяет **distinct** к получаемой таблице, то есть она отбрасывает повторяющиеся строки. Чтобы **union** не отбрасывал повторения нужно к нему добавлять слово **all**. На рис. 3.33 представлен пример корректного объединения таблиц student и prep по полю Фамилия.

Неверное объединение таблиц	Правильное объединение таблиц
select fam from student union select famPrep from prep	select fam from student union all select famPrep from prep

```
MySQL Command Line Client
mysql> select fam from student union all select famprep from prep;
+-------------+
| fam         |
+-------------+
| Иванов      |
| Иволгин     |
| Петров      |
| Сидоров     |
| Павлов      |
| Немцов      |
| Харитонова  |
| Парамонова  |
| Иванов      |
| Александров |
| Ситников    |
| Столыпин    |
| Крапивин    |
| Полушкин    |
| Полушкин    |
+-------------+
15 rows in set (0.01 sec)

mysql>
```

Рис. 3.33

Если же требуется выяснить какие фамилии существуют среди преподавателей и студентов, то следует выполнить запрос, представленный на рис. 3.34. Если с помощью union или union all несколько запросов соединяются в один, то инструкция сортировки **order by** ставиться в самом конце, чтобы отсортировать все результаты объединения

```
MySQL Command Line Client
mysql> select distinct fam from student fam union select famprep from prep
    -> order by fam;
+-------------+
| fam         |
+-------------+
| Александров |
| Иванов      |
| Иволгин     |
| Крапивин    |
| Немцов      |
| Парамонова  |
| Павлов      |
| Петров      |
| Полушкин    |
| Столыпин    |
| Ситников    |
| Сидоров     |
| Харитонова  |
+-------------+
13 rows in set (0.00 sec)

mysql>
```

Рис. 3.34